Mini-project 1 : Brazillian E-commerce

Project Overview

In this mini-project, you will explore and analyze a Brazilian e-commerce dataset to uncover insights and trends. You will perform data loading, cleaning, and various SQL queries to extract meaningful information about the e-commerce platform's operations and performance.

1. Load and Explore the Data :

```
!unzip "Brazilian E-Commerce Dive Deep using SQL.zip"
```

```
Archive:  Brazilian E-Commerce Dive Deep using SQL.zip
   creating: Brazilian E-Commerce Dive Deep using SQL/Brazilian E-Comm
   creating: Brazilian E-Commerce Dive Deep using SQL/Brazilian E-Comm
  inflating: Brazilian E-Commerce Dive Deep using SQL/Brazilian E-Comm
   creating: Brazilian E-Commerce Dive Deep using SQL/Brazilian E-Comm
  inflating: Brazilian E-Commerce Dive Deep using SQL/Brazilian E-Comm
   creating: Brazilian E-Commerce Dive Deep using SQL/Brazilian E-Comm
  inflating: Brazilian E-Commerce Dive Deep using SQL/Brazilian E-Comm
   creating: Brazilian E-Commerce Dive Deep using SQL/Brazilian E-Comm
  inflating: Brazilian E-Commerce Dive Deep using SQL/Brazilian E-Comm
   creating: Brazilian E-Commerce Dive Deep using SQL/Brazilian E-Comm
  inflating: Brazilian E-Commerce Dive Deep using SQL/Brazilian E-Comm
   creating: Brazilian E-Commerce Dive Deep using SQL/Brazilian E-Comm
  inflating: Brazilian E-Commerce Dive Deep using SQL/Brazilian E-Comm
   creating: Brazilian E-Commerce Dive Deep using SQL/Brazilian E-Comm
  inflating: Brazilian E-Commerce Dive Deep using SQL/Brazilian E-Comm
   creating: Brazilian E-Commerce Dive Deep using SQL/Brazilian E-Comm
  inflating: Brazilian E-Commerce Dive Deep using SQL/Brazilian E-Comm
  inflating: Brazilian E-Commerce Dive Deep using SQL/Brazilian E-Comm
  inflating: Brazilian E-Commerce Dive Deep using SQL/Brazilian E-Comm
```

```
import numpy as np
import pandas as pd
import sqlite3
from sqlalchemy import create_engine

base_path = 'Brazilian E-Commerce Dive Deep using SQL/Brazilian E-Com

df_olist_customers = pd.read_csv(base_path + 'olist_customers_dataset
df_olist_sellers = pd.read_csv(base_path + 'olist_sellers_dataset.csv
df_olist_order_reviews = pd.read_csv(base_path + 'olist_order_reviews_
df_olist_order_items = pd.read_csv(base_path + 'olist_order_items_dat
df_olist_products = pd.read_csv(base_path + 'olist_products_dataset.c
df_olist_geolocation = pd.read_csv(base_path + 'olist_geolocation_dat
df_product_category_name_translation = pd.read_csv(base_path + 'produ
df_olist_orders = pd.read_csv(base_path + 'olist_orders_dataset.csv/o
df_olist_order_payments = pd.read_csv(base_path + 'olist_order_paymen
```

```
engine = create_engine('sqlite://', echo=False)

df_olist_customers.to_sql("olist_customers", con=engine, index=False)
df_olist_sellers.to_sql("olist_sellers", con=engine, index=False)
df_olist_order_reviews.to_sql("olist_order_reviews", con=engine, inde
df_olist_order_items.to_sql("olist_order_items", con=engine, index=Fa
df_olist_products.to_sql("olist_products_dataset", con=engine, index=
df_olist_geolocation.to_sql("olist_geolocation", con=engine, index=Fa
df_product_category_name_translation.to_sql("product_category_name_tr
df_olist_orders.to_sql("olist_orders", con=engine, index=False)
df_olist_order_payments.to_sql("olist_order_payments", con=engine, in

print("Data loaded and SQL tables created successfully!")
```

```
Data loaded and SQL tables created successfully!
```

```
sql = '''
SELECT * FROM olist_customers
LIMIT 5
'''

df_sql = pd.read_sql_query(sql, con=engine)
df_sql
```

	customer_id	customer_unique_id	custo
0	06b8999e2fba1a1fbc88172c00ba8bc7	861eff4711a542e4b93843c6dd7febb0	
1	18955e83d337fd6b2def6b18a428ac77	290c77bc529b7ac935b93aa66c333dc3	
2	4e7b3e00288586ebd08712fdd0374a03	060e732b5b29e8181a18229c7b0b2b5e	
3	b2b6027bc5c5109e529d4dc6358b12c3	259dac757896d24d7702b9acbbff3f3c	
4	4f2d8ab171c80ec8364f7c12e35b23ad	345ecd01c38d18a9036ed96c73b8d066	

Next steps: Generate code with df_sql New interactive sheet

2. Create SQLite Database and Export DataFrames:

```
from sqlalchemy import create_engine

engine = create_engine('sqlite:///brazilian_ecommerce.db', echo=False

df_olist_customers.to_sql("olist_customers", con=engine, index=False,
df_olist_sellers.to_sql("olist_sellers", con=engine, index=False, if_
df_olist_order_reviews.to_sql("olist_order_reviews", con=engine, inde
df_olist_order_items.to_sql("olist_order_items", con=engine, index=Fa
df_olist_products.to_sql("olist_products", con=engine, index=False, i
df_olist_geolocation.to_sql("olist_geolocation", con=engine, index=Fa
```

```
df_product_category_name_translation.to_sql("product_category_transla
df_olist_orders.to_sql("olist_orders", con=engine, index=False, if_ex
df_olist_order_payments.to_sql("olist_order_payments", con=engine, in

print("Step 2 Complete: SQLite database created and tables populated!
```

```
Step 2 Complete: SQLite database created and tables populated!
```

3. Query 1: Count and Percentage of Orders Purchased in Jan 2018 with 5 Review Score:

```
sql_query_1 = '''
SELECT
    COUNT(*) AS total_january_orders,
    SUM(CASE WHEN review_score = 5 THEN 1 ELSE 0 END) AS five_star_or
    ROUND(AVG(CASE WHEN review_score = 5 THEN 100.0 ELSE 0.0 END), 2)
FROM olist_orders o
JOIN olist_order_reviews r ON o.order_id = r.order_id
WHERE o.order_purchase_timestamp >= '2018-01-01'
  AND o.order_purchase_timestamp <= '2018-01-31 23:59:59';
'''

df_jan_reviews = pd.read_sql_query(sql_query_1, con=engine)
df_jan_reviews
```

	total_january_orders	five_star_orders	percentage_five_star
0	7245	4097	56.55

4. Query 2: Customer Purchase Trend Year-on-Year:

```
sql_query_2 = '''
SELECT
    strftime('%Y', order_purchase_timestamp) AS purchase_year,
    COUNT(order_id) AS total_orders
FROM olist_orders
GROUP BY purchase_year
ORDER BY purchase_year;
'''

df_yearly_trend = pd.read_sql_query(sql_query_2, con=engine)
df_yearly_trend
```

	purchase_year	total_orders
0	2016	329
1	2017	45101
2	2018	54011

Next steps: Generate code with df_yearly_trend New interactive sheet

5. Query 3: Average Order Values of Customers:

```
sql_query_3 = '''
SELECT
    AVG(order_total) AS average_order_value
FROM (
    SELECT
        order_id,
        SUM(payment_value) AS order_total
    FROM olist_order_payments
    GROUP BY order_id
) AS order_totals;
'''

df_aov = pd.read_sql_query(sql_query_3, con=engine)
df_aov
```

	average_order_value
0	160.990267

6. Query 4: Top 5 Cities with Highest Revenue from 2016 to 2018:

```
sql_query_4 = '''
SELECT
    c.customer_city,
    SUM(p.payment_value) AS total_revenue
FROM olist_orders o
JOIN olist_order_payments p ON o.order_id = p.order_id
JOIN olist_customers c ON o.customer_id = c.customer_id
WHERE strftime('%Y', o.order_purchase_timestamp) BETWEEN '2016' AND '
GROUP BY c.customer_city
ORDER BY total_revenue DESC
LIMIT 5;
'''

df_top_cities = pd.read_sql_query(sql_query_4, con=engine)
df_top_cities
```

	customer_city	total_revenue
0	sao paulo	2203373.09
1	rio de janeiro	1161927.36
2	belo horizonte	421765.12
3	brasilia	354216.78
4	curitiba	247392.48

Next steps: Generate code with df_top_cities New interactive sheet

7. Query 5: State Wise Revenue Table Between 2016 to 2018:

```
sql_query_5 = '''
SELECT
    c.customer_state,
    SUM(p.payment_value) AS total_revenue
FROM olist_orders o
JOIN olist_order_payments p ON o.order_id = p.order_id
JOIN olist_customers c ON o.customer_id = c.customer_id
WHERE strftime('%Y', o.order_purchase_timestamp) BETWEEN '2016
GROUP BY c.customer_state
ORDER BY total_revenue DESC;
'''

df_state_revenue = pd.read_sql_query(sql_query_5, con=engine)
df_state_revenue
```

	customer_state	total_revenue
0	SP	5998226.96
1	RJ	2144379.69
2	MG	1872257.26
3	RS	890898.54
4	PR	811156.38
5	SC	623086.43
6	BA	616645.82
7	DF	355141.08
8	GO	350092.31
9	ES	325967.55
10	PE	324850.44
11	CE	279464.03
12	PA	218295.85
13	MT	187029.29
14	MA	152523.02
15	PB	141545.72
16	MS	137534.84
17	PI	108523.97
18	RN	102718.13
19	AL	96962.06
20	SE	75246.25
21	TO	61485.33
22	RO	60866.20
23	AM	27966.93
24	AC	19680.62
25	AP	16262.80
26	RR	10064.62

Next steps: Generate code with df_state_revenue New interactive sheet

8. Query 6: Top Successful Sellers in Terms of Goods Sold, Revenue, and Customer Count:

```
sql_query_6 = '''
SELECT
    oi.seller_id,
    COUNT(oi.order_item_id) AS total_goods_sold,
    ROUND(SUM(oi.price), 2) AS total_revenue,
    COUNT(DISTINCT o.customer_id) AS unique_customers,
    SUM(CASE WHEN r.review_score = 5 THEN 1 ELSE 0 END) AS five_star_
FROM olist_order_items oi
JOIN olist_orders o ON oi.order_id = o.order_id
JOIN olist_order_reviews r ON oi.order_id = r.order_id
GROUP BY oi.seller_id
ORDER BY total_revenue DESC
LIMIT 10;
'''

df_top_sellers = pd.read_sql_query(sql_query_6, con=engine)
df_top_sellers
```

	seller_id	total_goods_sold	total_revenue	uniq
0	4869f7a5dfa277a7dca6462dcf3b52b2	1148	228071.04	
1	53243585a1d6dc2643021fd1853d8905	408	220740.05	
2	4a3ca9315b744ce9f8e9374361493884	1984	200561.42	
3	fa1c13f2614d7b5c4749cbc52fecda94	582	192774.43	
4	7c67e1448b00f6e969d365cea6b010ab	1367	188017.85	
5	7e93a43ef30c4f03f38b393420bc753a	339	176201.88	
6	da8622b14eb17ae2831f4ac5b9dab84a	1568	161993.97	
7	7a67c85e85bb2ce8582c35f2203ad736	1166	141130.58	
8	1025f0e2d44d7041d6cf58b6550e0bfa	1431	139484.38	
9	955fee9216a65b617aa5c0531780ce60	1489	133948.81	

Next steps: Generate code with df_top_sellers | New interactive sheet

9. Query 7: Delivery Success Rate Across States:

```
sql_query_7 = '''
SELECT
    c.customer_state,
    COUNT(o.order_id) AS total_orders,
    SUM(CASE WHEN o.order_status = 'delivered' THEN 1 ELSE 0 E
    ROUND(SUM(CASE WHEN o.order_status = 'delivered' THEN 1.0
FROM olist_orders o
JOIN olist_customers c ON o.customer_id = c.customer_id
GROUP BY c.customer_state
ORDER BY success_rate_percentage DESC;
'''
```

```
df_delivery_success = pd.read_sql_query(sql_query_7, con=engin
df_delivery_success
```

	customer_state	total_orders	delivered_orders	success_rate_percen
0	AC	81	80	
1	AP	68	67	
2	ES	2033	1995	
3	MS	715	701	
4	AM	148	145	
5	TO	280	274	
6	RS	5466	5345	
7	RN	485	474	
8	MT	907	886	
9	PR	5045	4923	
10	MG	11635	11354	
11	SC	3637	3546	
12	DF	2140	2080	
13	PA	975	946	
14	SP	41746	40501	
15	GO	2020	1957	
16	PB	536	517	
17	PE	1652	1593	
18	BA	3380	3256	
19	PI	495	476	
20	AL	413	397	
21	RJ	12852	12350	
22	RO	253	243	
23	MA	747	717	
24	CE	1336	1279	
25	SE	350	335	
26	RR	46	41	

Next steps: Generate code with df_delivery_success New interactive sheet

10. Query 8: Preferred Form of Payment for Different Categories:

```
sql_query_8 = '''
WITH CategoryPaymentCounts AS (
    SELECT
        p.product_category_name,
        pay.payment_type,
        COUNT(*) AS count_usage
    FROM olist_products p
    JOIN olist_order_items oi ON p.product_id = oi.product_id
    JOIN olist_order_payments pay ON oi.order_id = pay.order_id
    WHERE p.product_category_name IS NOT NULL
    GROUP BY p.product_category_name, pay.payment_type
),
RankedPayments AS (
    SELECT
        product_category_name,
        payment_type,
        count_usage,
        RANK() OVER (PARTITION BY product_category_name ORDER BY coun
    FROM CategoryPaymentCounts
)
SELECT
    product_category_name,
    payment_type AS preferred_payment_method,
    count_usage
FROM RankedPayments
WHERE rank = 1
ORDER BY count_usage DESC;
'''

df_preferred_payment = pd.read_sql_query(sql_query_8, con=engine)
df_preferred_payment
```

	product_category_name	preferred_payment_method	co
0	cama_mesa_banho	credit_card	
1	beleza_saude	credit_card	
2	esporte_lazer	credit_card	
3	moveis_decoracao	credit_card	
4	informatica_acessorios	credit_card	
...	...	...	
69	portateis_cozinha_e_preparadores_de_alimentos	credit_card	
70	fashion_roupa_infanto_juvenil	credit_card	
71	pc_gamer	credit_card	
72	seguros_e_servicos	boleto	
73	seguros_e_servicos	credit_card	

74 rows × 3 columns

Next steps: Generate code with df_preferred_payment New interactive sheet

11. Query 9: Distance Between Cities:

```
import math
from sqlalchemy import text

def haversine(lat1, lon1, lat2, lon2):
    if None in (lat1, lon1, lat2, lon2):
        return None
    R = 6371.0
    phi1, phi2 = math.radians(lat1), math.radians(lat2)
    dphi = math.radians(lat2 - lat1)
    dlamb = math.radians(lon2 - lon1)
    a = math.sin(dphi/2)**2 + math.cos(phi1)*math.cos(phi2)*ma
    c = 2 * math.atan2(math.sqrt(a), math.sqrt(1-a))
    return round(R * c, 2)

raw_conn = engine.raw_connection()
raw_conn.create_function("haversine", 4, haversine)

with engine.connect() as conn:
    conn.execute(text('''
    CREATE TABLE IF NOT EXISTS zip_coords AS
    SELECT
        geolocation_zip_code_prefix AS zip,
        AVG(geolocation_lat) AS lat,
        AVG(geolocation_lng) AS lng
    FROM olist_geolocation
```

```
    GROUP BY zip
    '''))
    conn.commit()

sql_query_9 = '''
SELECT
    o.order_id,
    c.customer_city,
    s.seller_city,
    haversine(gc.lat, gc.lng, gs.lat, gs.lng) AS distance_km
FROM olist_orders o
JOIN olist_customers c ON o.customer_id = c.customer_id
JOIN olist_order_items oi ON o.order_id = oi.order_id
JOIN olist_sellers s ON oi.seller_id = s.seller_id
```